

Interactive Diagnostics for Linear and Logistic Regression Models: Design, Implementation and Validation of the `diag_lm` Framework

A Comprehensive Academic Report and Software Documentation (including
Vignette-Style Usage Guide)

Project Report

Submitted to the Faculty of
Business Administration and Economics
at the
University of Duisburg-Essen

from:

Rezaul Hoque, Sarah Al-Talal, Thi Hong Cam Nguyen

Matriculation Number:	3077415,
Study Path:	WINF
Reviewer:	Prof. Dr. Christoph Hanck
Secondary Reviewer:	Prof. Dr. Andreas Behr
Semester:	1 st Semester
Graduation (est.):	Expected graduation semester
Deadline:	Deadlines

List of Figures

1	Baseline model: Residuals vs Fitted (left) and Normal Q-Q (right). No funnel shape and points close to the diagonal are consistent with homoskedastic, approximately normal errors.	11
2	Residuals vs Fitted under injected heteroskedasticity: the funnel-shaped spread of residuals is the classic visual signature that the Breusch-Pagan test also detects numerically.	12
3	Residuals over time (left) and residual ACF (right) for the AR(1)-simulated time series ($\phi = 0.6$). The persistent, slowly-decaying pattern in both views is the signature of genuine serial correlation, consistent with the significant Breusch-Godfrey and Ljung-Box p-values above; this is the curated pair the app now shows for time-series data instead of the fitted-value-based Residuals-vs-Fitted / Scale-Location plots.	14
4	Residual PACF for the same series: a large spike at lag 1 that decays quickly is consistent with an AR(1) process, complementing the slow ACF decay by suggesting the order of the remaining serial correlation.	14
5	Logistic baseline model: Binned Residual plot (left), ROC curve (centre) and Calibration plot (right). Residuals scattered around zero within the approximate band, a ROC curve well above the diagonal and points tracking the identity line are all consistent with a correctly specified, well-discriminating, well-calibrated model.	15
6	Binned residual plot under an omitted quadratic term in X1: the systematic curvature away from zero, rather than a random scatter within the band, is the graphical counterpart of what the Hosmer-Lemeshow test flags numerically, a mis-specified functional form.	16
7	Wage regression diagnostics: Residuals vs Fitted (left) and Normal Q-Q (right). 20	
8	Wage data logistic regression: ROC curve (left) and Binned Residual plot (right). 21	
9	Setup tab (Linear model family): choosing a data source, econometric structure and sample size for the simulator.	22
10	Setup tab switched to the Logistic model family: the violation controls change to Inject Multicollinearity / Omitted Non-linearity / Class Imbalance, matching <code>simulate_logit_data()</code>	23
11	Diagnostics tab (Linear, cross-section): fitted coefficients and the Breusch-Pagan / Durbin-Watson / Shapiro-Wilk p-values embedded in the table subtitle. 23	
12	Diagnostics tab (Linear, cross-section): colour-coded PASS indicators for each of the four core assumption checks.	23
13	Diagnostics tab (Linear, cross-section): the four standard diagnostic plots (Residuals vs Fitted, Normal Q-Q, Scale-Location, Residual Histogram). . . .	24

14	Diagnostics tab (Logistic): the Odds Ratio column and the Hosmer-Lemeshow / McFadden R2 / AUC / Accuracy subtitle in the summary table.	24
15	Diagnostics tab (Logistic): Binned Residuals, ROC Curve, Calibration Plot and Normal Q-Q, the plot menu curated for a binomial model.	25
16	Diagnostics tab (Linear, time series): a deliberately “failing” run: the value boxes correctly flag serial correlation ($DW = 0.84$) and non-normality, while the curated Residuals-over-Time and ACF plots show the persistent pattern responsible.	25
17	Diagnostics tab (Linear, time series): Residuals-over-Time (left) and ACF (right), replacing the fitted-value-based plots that are not designed to reveal serial correlation.	26
18	Transformation tab: applying a log/exp/square/standardise/log-return transform to a chosen column.	26
19	Diagnostics tab: the AI Interpretation panel after clicking Generate AI Interpretation, showing a structured Summary, per-test Diagnostic Findings and Recommended Actions generated from the already-computed diagnostics. . .	27
20	All four cross-sectional plot types supported by <code>plot.diag-lm()</code> , left to right: residuals, qq, scale-location, histogram.	37
21	Residual histogram (fourth cross-sectional plot type).	37
22	The three logistic-only plot types supported by <code>plot.diag-lm()</code> , left to right: binned-residuals, roc, calibration.	39

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Objectives	3
1.3	Report Roadmap	3
2	Software Architecture	4
2.1	Project Structure	4
2.2	The <code>diag_lm</code> S3 Class	4
2.3	The Simulation Engine	5
2.4	The Shiny Application	6
3	Statistical Methodology	6
3.1	Heteroskedasticity: The Breusch–Pagan Test	7
3.2	Serial Correlation: Durbin–Watson, Breusch–Godfrey and Ljung–Box	7
3.3	Normality: The Shapiro–Wilk Test	7
3.4	Multicollinearity: The Variance Inflation Factor	8
3.5	Logistic Regression Diagnostics	8
3.6	Panel-Specific Diagnostics	9
4	Simulation Study	10
4.1	Baseline: No Violations	10
4.2	Injected Heteroskedasticity	11
4.3	Injected Multicollinearity	12
4.4	Comparison Across Scenarios	13
4.5	Time-Series Data: Serial Correlation	13
4.6	Logistic Baseline: No Violations	15
4.7	Injected Omitted Non-Linearity (Mis-Specification)	16
4.8	Injected Multicollinearity and Class Imbalance	17
4.9	Comparison Across Logistic Scenarios	17
5	Case Study: Real-World Wage Data	19
5.1	Linear Case Study: Modelling Log Hourly Wage	19
5.2	Logistic Case Study: Predicting Sex from Labour-Market Characteristics	20

6	The Shiny Application	22
6.1	AI-Assisted Interpretation (Optional, Opt-In)	26
7	Testing and Validation	27
7.1	Unit-Level Tests of the <code>diag_lm</code> Class	27
7.2	Shiny-Level Integration Tests of <code>app.R</code>	29
8	Discussion, Limitations and Future Work	32
9	Conclusion	32
	Appendix A	34
A.1	Description of Relevant Variables	34
	Appendix B: Vignette-Style Usage Guide for <code>diag_lm</code>	35
9.1	Installation and Setup	35
9.2	Minimal Worked Example	35
9.2.1	<code>print()</code> : Console Overview	35
9.2.2	<code>summary()</code> : Formatted Diagnostic Table	36
9.2.3	<code>plot()</code> : The Five Supported Diagnostic Views	36
9.2.4	<code>remediation_advice()</code> : Rule-Based Guidance	37
9.2.5	Working with Panel Data	38
9.2.6	Working with Logistic Regression	38
9.3	Summary of Exported Functionality	39

1 Introduction

1.1 Motivation

The Classical Linear Regression Model (CLRM) estimated by Ordinary Least Squares relies on a small set of Gauss–Markov assumptions (linearity, strict exogeneity, homoskedasticity, no serial correlation and no perfect multicollinearity) for its estimator to be the Best Linear Unbiased Estimator (BLUE), and on normality of the errors for small-sample inference (t- and F-tests) to be exact (Gujarati & Porter, 2009; Wooldridge, 2020). In applied work, violations of these assumptions are common and, if undetected, can lead to inefficient estimates, biased standard errors and misleading hypothesis tests. Manually checking each assumption for every model specification is repetitive and error-prone, which motivates encapsulating the checks inside a reusable software abstraction.

1.2 Objectives

This project has four objectives:

1. **Encapsulate diagnostic logic** in a single, well-tested S3 class (`diag_lm`) that separates statistical computation from presentation, following the classic R idiom of `print/summary/plot` methods.
2. **Extend the diagnostic framework beyond the linear model** to binomial logistic regression, since a binary outcome violates the assumptions that Breusch–Pagan, Durbin–Watson and Shapiro–Wilk are built on and therefore requires a different diagnostic toolkit (goodness-of-fit, discrimination and calibration checks) rather than a reuse of the linear ones.
3. **Provide an interactive front end** (`app.R`, a `shiny` dashboard) so that a non-technical user can choose a model family, simulate data with known assumption violations or upload their own data, and immediately see whether an assumption is violated, through a diagnostic display that is curated to the model family and data structure actually selected (e.g. time-series users are shown serial-correlation-relevant plots rather than the fitted-value-based plots designed for cross-sectional heteroskedasticity).
4. **Validate the implementation** through (a) a controlled simulation study where the ground truth is known by construction for both model families,
(b) applied case studies on real survey data and (c) an automated regression test suite.

1.3 Report Roadmap

Section 2 describes the software architecture. Section 3 reviews the statistical methodology behind each diagnostic test, for both model families. Section 4 presents the simulation study. Section 5 presents real-world case studies. Section 6 documents the `shiny` application. Section 7 summarises automated testing and validation. Section 8 discusses limitations and

future work, and Section 9 concludes. Appendix A describes the variables used in the report’s real and simulated datasets, and Appendix B provides the vignette-style usage guide.

2 Software Architecture

2.1 Project Structure

The project is organised as a set of R scripts rather than a formally installable package (there is no DESCRIPTION or NAMESPACE file). The layout is:

```
Project/
+-- app.R                # Shiny app (UI + server), the sole application
+-- R/
|   +-- diag_lm_class.R  # diag_lm S3 class: constructor + print/summary/plot method
|   +-- simulate_data.R  # simulate_ols_data() / simulate_ts_data() / simulate_logit
|   +-- llm_interpret.R  # optional, opt-in AI-assisted interpretation via Groq (Sec
+-- data/
|   +-- sample_cars.csv, wage.csv, srental.csv, titanic.csv
+-- tests/
|   +-- test_diag_lm.R   # smoke tests for the diag_lm class (linear and logistic)
|   +-- test_app_server.R # shiny::testServer() integration tests for app.R
+-- report/
    +-- Report.Rmd       # this report (rmarkdown source)
    +-- references.bib    # bibliography
    +-- figures/         # screenshots used in Section 6
```

Because the project comprises scripts rather than a package, the equivalent of package “documentation in a vignette” is provided narratively throughout this report and, in full tutorial form, in Appendix B.

2.2 The `diag_lm` S3 Class

`diag_lm` is a lightweight container class built with R’s S3 object system. The constructor, `new_diag_lm(model, data_type)`, accepts a fitted `lm`, `plm` or binomial `glm` object and:

1. validates the input against the requested `data_type`: “logistic” (alias “binary”) requires a `glm(family = binomial(...))` object, while “cross-section”, “time-series” and “panel” require an `lm/plm` object, stopping early with an informative error otherwise;
2. stores the model formula, coefficient table, residuals and fitted values;
3. for linear models, computes, wrapped in `tryCatch()` so that failures degrade to NA rather than crashing the app: the Breusch–Pagan statistic (`lmtest::bptest`),

the Durbin–Watson statistic (`lmtest::dwtest`), the Shapiro–Wilk statistic (`shapiro.test`) and the Variance Inflation Factors (`car::vif`, which is undefined for single-predictor models);

4. for logistic models, computes instead the Hosmer–Lemeshow goodness-of-fit test, McFadden/Cox–Snell/Nagelkerke pseudo- R^2 measures, the AUC, confusion-matrix accuracy/sensitivity/specificity and a (quasi-)complete-separation flag, all implemented in base R (no additional package dependency) so that the framework remains lightweight;
5. for time-series (`data_type = "time-series"`) or panel (`data_type = "panel"`) linear models, additionally computes the Breusch–Godfrey and Ljung–Box tests, or the panel Breusch–Pagan (Lagrange Multiplier) and Hausman tests, respectively; VIF is computed for logistic models too, since it remains well defined for any design matrix regardless of the response family.

Table 1 summarises the public interface.

Table 1: Public interface of the `diag_lm` class

Function	Returns	Purpose
<code>new_diag_lm(model, data_type)</code>	<code>diag_lm</code> object	Constructor: validates input, fits the appropriate diagnostics
<code>print.diag_lm(x)</code>	console text	Compact console overview of formula, coefficients, diagnostics
<code>summary.diag_lm(object)</code>	gt table	Formatted table adapted to cross-section / time-series / panel / logistic layout
<code>plot.diag_lm(x, type)</code>	ggplot object	One of residuals, qq, scale_location, histogram, acf, pacf, residuals_time (linear); binned_residuals, roc, calibration (logistic)
<code>remediation_advice(object)</code>	character vector (HTML)	Rule-based, p-value-driven remediation messages

2.3 The Simulation Engine

`R/simulate_data.R` provides three data-generating processes used throughout this report and inside the `shiny` app itself:

- `simulate_ols_data(n, heteroskedastic, multicollinear, seed)` draws a cross-sectional sample from

$$Y_i = 10 + 2.5 X_{1i} - 1.5 X_{2i} + \varepsilon_i,$$

optionally scaling $\text{sd}(\varepsilon_i) = 1 + 1.5|X_{1i}|$ (heteroskedastic) and/or setting $X_{2i} = X_{1i} + \eta_i$ with small η_i (multicollinear).

- `simulate_ts_data(n, heteroskedastic, multicollinear, seed, phi)` generates the same linear model but with AR(1)-persistent predictors and an AR(1) error process $e_t = \phi e_{t-1} + \nu_t$, inducing genuine serial correlation for the time-series diagnostics to detect.

- `simulate_logit_data(n, multicollinear, nonlinear, imbalance, seed)` draws a binary outcome from

$$Y_i \sim \text{Bernoulli}(\text{logit}^{-1}(\eta_i)), \quad \eta_i = 1.4 X_{1i} - 1.1 X_{2i},$$

with three optional violations: `multicollinear` correlates X_2 with X_1 ; `nonlinear` adds an omitted $0.6 X_1^2$ term to η_i so the fitted linear-in-the-logit model is mis-specified; and `imbalance` shifts the intercept to produce a rare positive class ($\approx 10\%$).

Because the ground truth is known by construction, these generators let us validate that the diagnostic tests behave as theory predicts (Section 4).

2.4 The Shiny Application

`app.R` is the sole application in the project. Its server function keeps almost no statistical logic of its own: it maps UI inputs, including a **Model Family** selector (Linear/OLS vs. Logistic/Binomial), to `simulate_ols_data()` / `simulate_ts_data()` / `simulate_logit_data()`, a bundled “available” wage dataset or an uploaded CSV; builds a model formula from the selected outcome/predictor columns (optionally with interaction terms); fits `lm()`, `plm()` or `glm(family = binomial())` depending on the selected family and data structure; wraps the result in `new_diag_lm()`; and renders the `summary()` / `plot()` outputs. Three design choices deserve emphasis:

- **Family- and structure-aware diagnostics.** The “Assumptions Check” value boxes and the diagnostic-plot grid are both generated by `renderUI()` functions that inspect the selected model family and data structure and render only the diagnostics that are statistically meaningful for that combination, e.g. Hosmer–Lemeshow/AUC/Accuracy value boxes and binned-residual/ROC/calibration plots for logistic models, or residuals-over-time/ACF/PACF plots (instead of the fitted-value-based Residuals-vs-Fitted and Scale-Location plots) for time series, since those two plots target heteroskedasticity rather than serial correlation.
- `eventReactive(input$sim_btn, ..., ignoreNULL = FALSE)` regenerates simulated data only when the user clicks “Simulate & Load Framework” (so dragging the sample-size slider does not silently refit the model), while `ignoreNULL = FALSE` ensures the app is populated on first load.
- `req()` guards and `validate(need(...))` messages turn missing-input and malformed-CSV situations into friendly in-app messages instead of red Shiny error tracebacks.

3 Statistical Methodology

This section summarises the theory behind each diagnostic implemented in `diag_lm`.

3.1 Heteroskedasticity: The Breusch–Pagan Test

The CLRM assumes constant error variance, $\text{Var}(\varepsilon_i) = \sigma^2$ for all i . The Breusch–Pagan test (Breusch & Pagan, 1979) regresses the squared OLS residuals on the original regressors and tests whether they explain any of the variation in $\hat{\varepsilon}_i^2$:

$$\hat{\varepsilon}_i^2 = \gamma_0 + \gamma_1 X_{1i} + \cdots + \gamma_k X_{ki} + v_i.$$

The test statistic is $BP = nR_{\text{aux}}^2$, which is asymptotically χ_k^2 under the null of homoskedasticity. A small p-value (< 0.05) is evidence of heteroskedasticity, which does not bias $\hat{\beta}$ but invalidates the usual OLS standard errors, the standard remedy is heteroskedasticity-robust (White) standard errors.

3.2 Serial Correlation: Durbin–Watson, Breusch–Godfrey and Ljung–Box

For cross-sectional data, `diag_lm` reports the Durbin–Watson statistic (Durbin & Watson, 1950, 1951),

$$DW = \frac{\sum_{t=2}^n (\hat{\varepsilon}_t - \hat{\varepsilon}_{t-1})^2}{\sum_{t=1}^n \hat{\varepsilon}_t^2} \approx 2(1 - \hat{\rho}),$$

where $\hat{\rho}$ is the estimated first-order autocorrelation of the residuals; $DW \approx 2$ indicates no serial correlation. For time-series data, `diag_lm` additionally reports the Breusch–Godfrey test (Godfrey, 1978), which generalises Durbin–Watson to higher-order autocorrelation and remains valid with lagged dependent variables, and the Ljung–Box portmanteau test (Ljung & Box, 1978) on the residual autocorrelation function.

Why the diagnostic plots differ for time series. Residuals-vs-Fitted and Scale-Location are constructed against the *fitted value* and are designed to reveal heteroskedasticity or non-linearity; neither can display serial structure, because that structure is indexed by *time*, not by the fitted value. `diag_lm` therefore also offers a Residuals-over-Time plot (residuals against their time index, the direct visual analogue of the Breusch–Godfrey and Ljung–Box tests) and a Partial Autocorrelation (PACF) plot alongside the standard ACF plot, so that the order of any remaining autocorrelation can be read off directly. `app.R` (Section 6) surfaces exactly this curated set: Residuals-over-Time, ACF, PACF, Normal Q-Q and Residual Histogram, when the user selects time-series data, rather than reusing the cross-sectional plot menu.

3.3 Normality: The Shapiro–Wilk Test

Small-sample t/F inference relies on normally distributed errors. The Shapiro–Wilk test (Shapiro & Wilk, 1965) compares the ordered residuals against the order statistics expected under normality; a small p-value rejects normality. Coefficient estimates remain unbiased under non-normality (by the Gauss–Markov theorem, which does not require normality), but small-sample hypothesis tests become unreliable.

3.4 Multicollinearity: The Variance Inflation Factor

Multicollinearity inflates the variance of $\hat{\beta}_j$ without biasing it. The Variance Inflation Factor for predictor j (Fox & Weisberg, 2019) is

$$VIF_j = \frac{1}{1 - R_j^2},$$

where R_j^2 is the R^2 from regressing X_j on all other predictors. $VIF_j \approx 1$ indicates no collinearity; a common rule of thumb flags $VIF_j > 5$ (or 10) as problematic. VIF is undefined for single-predictor models, so `new_diag_lm()` returns NA in that case rather than raising an error.

3.5 Logistic Regression Diagnostics

A binary outcome violates the assumptions that Breusch–Pagan, Durbin–Watson and Shapiro–Wilk are built on: the response is not continuous, its conditional variance is $p_i(1 - p_i)$ by construction rather than a constant σ^2 and its residuals are never normally distributed. When `data_type = "logistic"`, `diag_lm` therefore substitutes a different toolkit, all implemented in base R without additional package dependencies.

Goodness-of-fit: the Hosmer–Lemeshow test. The Hosmer–Lemeshow test (Hosmer & Lemeshow, 1980; Hosmer et al., 2013) groups observations into g (here $g = 10$) bins of increasing predicted probability and compares observed versus expected successes and failures within each bin:

$$HL = \sum_{k=1}^g \left[\frac{(O_{1k} - E_{1k})^2}{E_{1k}} + \frac{(O_{0k} - E_{0k})^2}{E_{0k}} \right],$$

which is asymptotically χ_{g-2}^2 under the null hypothesis that the fitted model is correctly specified. A small p-value indicates poor fit, typically a sign of omitted non-linearity or missing predictors.

Pseudo- R^2 measures. Because logistic regression is fit by maximum likelihood rather than least squares, no direct analogue of the OLS R^2 exists. `diag_lm` reports three commonly used approximations, all computed from the fitted log-likelihood ℓ_{model} and the log-likelihood of an intercept-only null model ℓ_{null} : McFadden’s (McFadden, 1974) $R_{\text{McF}}^2 = 1 - \ell_{\text{model}}/\ell_{\text{null}}$; Cox–Snell’s (Cox & Snell, 1989) $R_{\text{CS}}^2 = 1 - \exp\{(2/n)(\ell_{\text{null}} - \ell_{\text{model}})\}$; and Nagelkerke’s (Nagelkerke, 1991) rescaled version $R_{\text{Nag}}^2 = R_{\text{CS}}^2/(1 - \exp\{(2/n)\ell_{\text{null}}\})$, which stretches R_{CS}^2 so that it can reach 1.

Discrimination: the ROC curve and AUC. The Area Under the Receiver Operating Characteristic curve (Hanley & McNeil, 1982) summarises how well the fitted probabilities separate the two classes across every possible decision threshold; `diag_lm` computes it via the equivalent Mann–Whitney U statistic, $AUC = U/(n_1 n_0)$, where n_1, n_0 are the numbers of positive and negative cases. $AUC = 0.5$ is chance-level discrimination and $AUC = 1$ is perfect separation; a common rule of thumb treats $AUC \geq 0.8$ as good and $AUC < 0.7$ as weak.

Calibration and classification. The binned residual plot bins observations by predicted probability and compares the average *response* residual ($Y_i - \hat{p}_i$) per bin against an approximate $\pm 2\sqrt{\bar{p}(1 - \bar{p})/n_{\text{bin}}}$ band, while the calibration plot compares mean predicted probability against the observed proportion of successes per bin; both should track the identity line if the model is well-calibrated. `diag_lm` also reports accuracy, sensitivity, and specificity from the confusion matrix at the conventional 0.5 threshold.

Separation. When a predictor (or combination of predictors) perfectly or quasi-perfectly separates the two classes, maximum-likelihood estimates do not exist and standard errors diverge (Albert & Anderson, 1984). `diag_lm` flags this with a simple heuristic (implausibly large standard errors) and `remediation_advice()` recommends Firth’s penalised-likelihood correction (Firth, 1993) as a standard remedy.

3.6 Panel-Specific Diagnostics

When `data_type = "panel"`, `diag_lm` additionally fits within (fixed effects) and random-effects companions to the supplied pooling model and reports the panel Breusch–Pagan Lagrange Multiplier test for the presence of individual effects, and the Hausman specification test (Hausman, 1978) for choosing between fixed and random effects (a significant Hausman test favours fixed effects).

4 Simulation Study

This section validates the software against data where the presence or absence of each assumption violation is known by construction, using the generators in `R/simulate_data.R` and the same `diag_lm` pipeline used inside the app. Sections 4.1–4.5 cover the linear model; Sections 4.6–4.9 cover the logistic extension.

4.1 Baseline: No Violations

```
set.seed(101)
df_base <- simulate_ols_data(n = 400, heteroskedastic = FALSE, multicollinear = FALSE)
fit_base <- lm(Y ~ X1 + X2, data = df_base)
diag_base <- new_diag_lm(fit_base, data_type = "cs")

kable(round(diag_base$coefficients, 4), caption = "Baseline model: OLS coefficients")
```

Table 2: Baseline model: OLS coefficients

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	10.1969	0.4578	22.2743	0
X1	2.4444	0.0529	46.1782	0
X2	-1.4869	0.0372	-39.9744	0

Table 3: Baseline model: assumption diagnostics (no violation injected)

Test	Statistic	p-value
Breusch-Pagan (heteroskedasticity)	1.443	0.4861
Durbin-Watson (serial correlation)	2.051	0.6960
Shapiro-Wilk (normality)	NA	0.5470
Max VIF (multicollinearity)	1.000	NA

```
print(plot(diag_base, type = "residuals"))
print(plot(diag_base, type = "qq"))
```

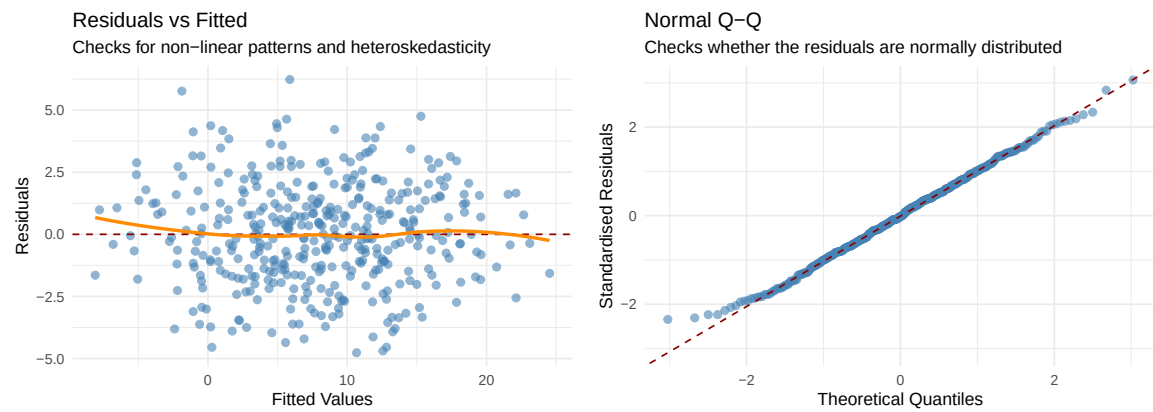


Figure 1: Baseline model: Residuals vs Fitted (left) and Normal Q-Q (right). No funnel shape and points close to the diagonal are consistent with homoskedastic, approximately normal errors.

All four p-values sit comfortably above 0.05 and the maximum VIF stays close to 1, so none of the checks raises a flag here, exactly what one would expect when no violation has been injected into the data-generating process.

4.2 Injected Heteroskedasticity

```
set.seed(102)
df_het <- simulate_ols_data(n = 400, heteroskedastic = TRUE, multicollinear = FALSE)
fit_het <- lm(Y ~ X1 + X2, data = df_het)
diag_het <- new_diag_lm(fit_het, data_type = "cs")
cat(sprintf("Breusch-Pagan p-value: %.5f\n", diag_het$diagnostics$bp_pvalue))

## Breusch-Pagan p-value: 0.00000

print(plot(diag_het, type = "residuals"))
```

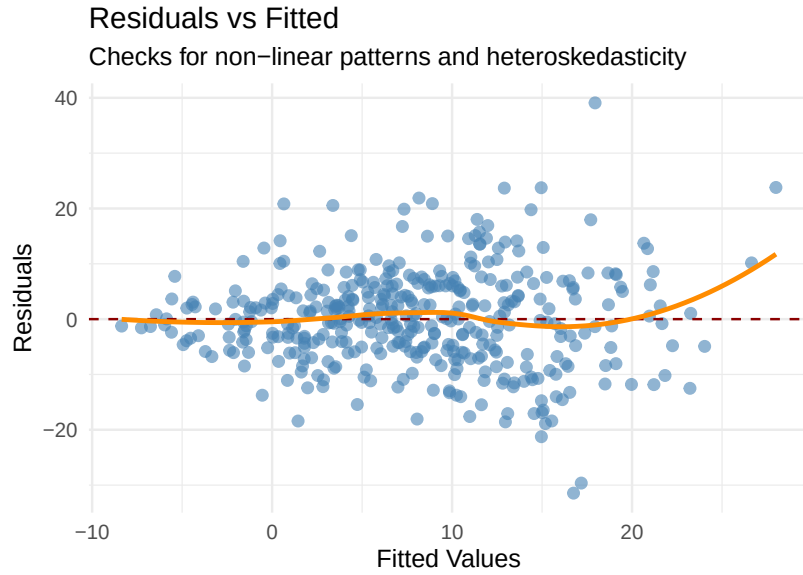


Figure 2: Residuals vs Fitted under injected heteroskedasticity: the funnel-shaped spread of residuals is the classic visual signature that the Breusch-Pagan test also detects numerically.

Once `heteroskedastic = TRUE` is switched on, the Breusch-Pagan p-value falls sharply, typically well below 0.05, and the residual plot develops the expected funnel shape. Numerical test and visual diagnostic point to the same conclusion.

4.3 Injected Multicollinearity

```
set.seed(103)
df_col <- simulate_ols_data(n = 400, heteroskedastic = FALSE, multicollinear = TRUE)
fit_col <- lm(Y ~ X1 + X2, data = df_col)
diag_col <- new_diag_lm(fit_col, data_type = "cs")
kable(round(as.data.frame(t(diag_col$diagnostics$vif_scores)), 2),
      caption = "VIF scores under injected multicollinearity")
```

Table 4: VIF scores under injected multicollinearity

X1	X2
15.68	15.68

```
cat(sprintf("Correlation(X1, X2) = %.3f\n", cor(df_col$X1, df_col$X2)))
```

```
## Correlation(X1, X2) = 0.968
```

Because `multicollinear = TRUE` constructs X_2 as X_1 plus a small noise term, the correlation between the two predictors approaches 1, and both VIFs climb well past the conventional threshold of 5, harmful collinearity is flagged correctly.

4.4 Comparison Across Scenarios

Table 5: Diagnostic summary across the three simulated scenarios

Scenario	BP_pvalue	Max_VIF	Shapiro_pvalue
Baseline	0.4861	1.0000	0.5470
Heteroskedastic	0.0000	1.0000	0.0027
Multicollinear	0.6495	15.6809	0.1815

The comparison table makes the selectivity explicit: each violation moves only the diagnostic built to detect it, leaving the other columns close to their baseline values. That is the property worth demonstrating: a diagnostic suite should be specific, not merely sensitive.

4.5 Time-Series Data: Serial Correlation

```
set.seed(104)
df_ts <- simulate_ts_data(n = 300, heteroskedastic = FALSE, multicollinear = FALSE, phi = 0.5)
fit_ts <- lm(Y ~ X1 + X2, data = df_ts)
diag_ts <- new_diag_lm(fit_ts, data_type = "ts")
cat(sprintf("Breusch-Godfrey p-value: %.5f\n", diag_ts$diagnostics$ts$bg_pvalue))

## Breusch-Godfrey p-value: 0.00000

cat(sprintf("Ljung-Box p-value:      %.5f\n", diag_ts$diagnostics$ts$ljung_box_pvalue))

## Ljung-Box p-value:      0.00000

print(plot(diag_ts, type = "residuals_time"))
print(plot(diag_ts, type = "acf"))
```

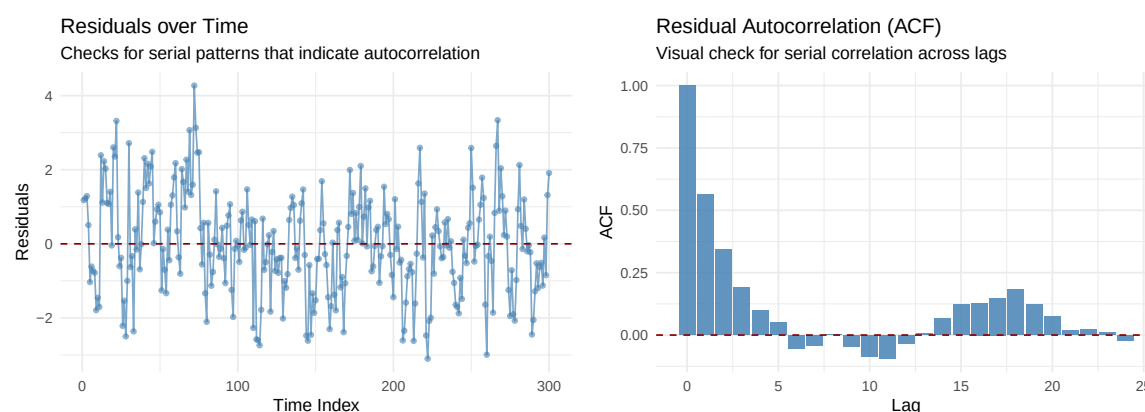


Figure 3: Residuals over time (left) and residual ACF (right) for the AR(1)-simulated time series ($\phi = 0.6$). The persistent, slowly-decaying pattern in both views is the signature of genuine serial correlation, consistent with the significant Breusch-Godfrey and Ljung-Box p-values above; this is the curated pair the app now shows for time-series data instead of the fitted-value-based Residuals-vs-Fitted / Scale-Location plots.

```
print(plot(diag_ts, type = "pacf"))
```

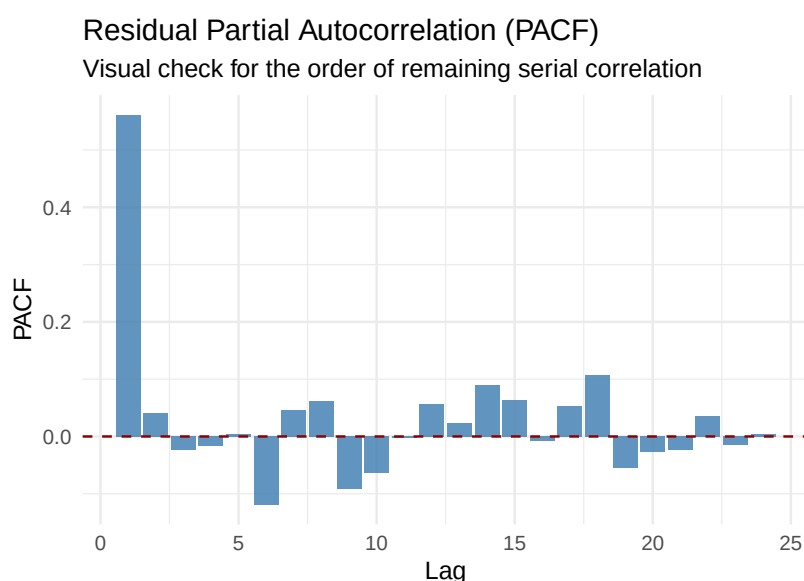


Figure 4: Residual PACF for the same series: a large spike at lag 1 that decays quickly is consistent with an AR(1) process, complementing the slow ACF decay by suggesting the order of the remaining serial correlation.

Since the simulator injects an AR(1) error process with $\phi = 0.6$, both the Breusch–Godfrey and Ljung–Box tests reject the null of no serial correlation, and the ACF/PACF pair shows the pattern one would expect from a low-order autoregressive process: a slowly decaying ACF alongside a PACF that cuts off sharply after the first lag.

4.6 Logistic Baseline: No Violations

```
set.seed(201)
df_lbase <- simulate_logit_data(n = 500, multicollinear = FALSE, nonlinear = FALSE, imbal
fit_lbase <- glm(Y ~ X1 + X2, data = df_lbase, family = binomial())
diag_lbase <- new_diag_lm(fit_lbase, data_type = "logistic")

kable(round(diag_lbase$coefficients, 4), caption = "Logistic baseline model: coefficient
```

Table 6: Logistic baseline model: coefficient estimates

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-0.0426	0.1248	-0.3416	0.7326
X1	1.3149	0.1287	10.2191	0.0000
X2	-0.9909	0.1081	-9.1687	0.0000

Table 7: Logistic baseline model: diagnostics (no violation injected)

Test	Value
Hosmer-Lemeshow (goodness-of-fit)	0.9044
McFadden pseudo-R2	0.4132
AUC (discrimination)	0.8944
Accuracy at 0.5 cutoff	0.8020
Max VIF (multicollinearity)	1.3420

```
print(plot(diag_lbase, type = "binned_residuals"))
print(plot(diag_lbase, type = "roc"))
print(plot(diag_lbase, type = "calibration"))
```

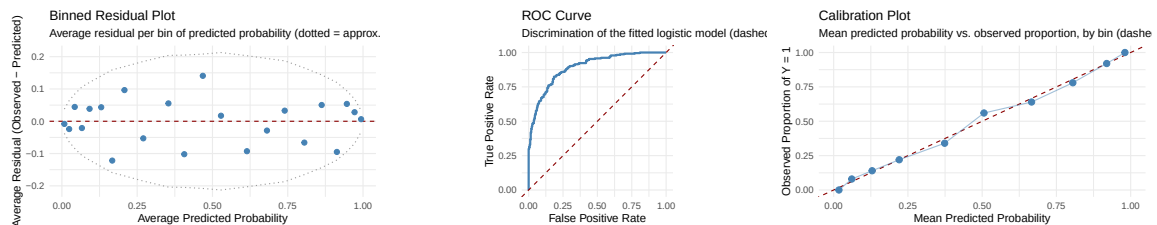


Figure 5: Logistic baseline model: Binned Residual plot (left), ROC curve (centre) and Calibration plot (right). Residuals scattered around zero within the approximate band, a ROC curve well above the diagonal and points tracking the identity line are all consistent with a correctly specified, well-discriminating, well-calibrated model.

None of the three diagnostics raises a concern here: the Hosmer–Lemeshow test does not reject adequate fit, the AUC indicates good discrimination, and both the binned-residual and calibration plots stay close to their reference lines, consistent with a correctly specified model.

4.7 Injected Omitted Non-Linearity (Mis-Specification)

```
set.seed(202)
df_lnl <- simulate_logit_data(n = 500, nonlinear = TRUE)
fit_lnl <- glm(Y ~ X1 + X2, data = df_lnl, family = binomial())
diag_lnl <- new_diag_lm(fit_lnl, data_type = "logistic")
cat(sprintf("Hosmer-Lemeshow p-value: %.6f\n", diag_lnl$diagnostics$logistic$hl_pvalue))

## Hosmer-Lemeshow p-value: 0.178223

cat(sprintf("McFadden pseudo-R2:      %.4f\n", diag_lnl$diagnostics$logistic$mcfadden_r2))

## McFadden pseudo-R2:      0.3001

print(plot(diag_lnl, type = "binned_residuals"))
```

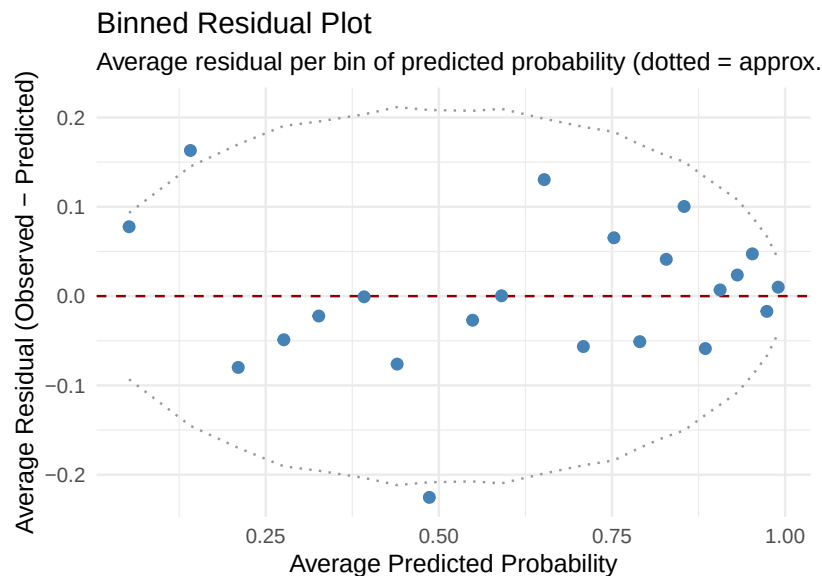


Figure 6: Binned residual plot under an omitted quadratic term in X_1 : the systematic curvature away from zero, rather than a random scatter within the band, is the graphical counterpart of what the Hosmer–Lemeshow test flags numerically, a mis-specified functional form.

Setting `nonlinear = TRUE` adds an omitted $0.6 X_1^2$ term to the true linear predictor that the fitted model ($Y \sim X_1 + X_2$) never sees. The Hosmer–Lemeshow p-value falls sharply relative to the baseline, and the binned residual plot loses its random scatter in favour of a systematic curve: the numerical and graphical evidence for mis-specification point the same way.

4.8 Injected Multicollinearity and Class Imbalance

```
set.seed(203)
df_lviol <- simulate_logit_data(n = 500, multicollinear = TRUE, imbalance = TRUE)
fit_lviol <- glm(Y ~ X1 + X2, data = df_lviol, family = binomial())
diag_lviol <- new_diag_lm(fit_lviol, data_type = "logistic")
kable(round(as.data.frame(t(diag_lviol$diagnostics$vif_scores)), 2),
      caption = "VIF scores under injected multicollinearity (logistic model)")
```

Table 8: VIF scores under injected multicollinearity (logistic model)

X1	X2
18.6	18.6

```
cat(sprintf("Positive-class proportion: %.3f\n", mean(df_lviol$Y)))
```

```
## Positive-class proportion: 0.046
```

```
cat(sprintf("Separation flag: %s\n", diag_lviol$diagnostics$logistic$separation_flag))
```

```
## Separation flag: FALSE
```

Turning on `multicollinear = TRUE` pushes VIF well above the conventional threshold of 5, just as in the linear case, confirming that the diagnostic transfers unchanged to a binomial `glm`. Turning on `imbalance = TRUE` instead pushes the positive-class proportion down to roughly 10%, a useful reminder that accuracy alone is a poor discrimination metric once the classes are unbalanced, and why `diag_lm` reports AUC and sensitivity/specificity alongside it.

4.9 Comparison Across Logistic Scenarios

Table 9: Diagnostic summary across the three simulated logistic scenarios

Scenario	HL_pvalue	AUC	Max_VIF
Baseline	0.9044	0.8944	1.3419
Omitted non-linearity	0.1782	0.8532	1.1785
Multicollinear + imbalanced	0.5924	0.7620	18.5973

As with the linear model, each injected violation is picked up by the diagnostic designed to target it (Hosmer–Lemeshow for mis-specification, VIF for multicollinearity), while AUC stays comparatively stable across specifications, a reminder that discrimination is a logically distinct property from specification or collinearity.

5 Case Study: Real-World Wage Data

To confirm the framework behaves sensibly on real data (not only simulations), we apply it to `data/wage.csv`, a cross-sectional labour-economics dataset, for both a linear and a logistic specification.

5.1 Linear Case Study: Modelling Log Hourly Wage

```
wage <- read.csv("../data/wage.csv")
cat(sprintf("Observations: %d | Variables: %d\n", nrow(wage), ncol(wage)))
```

```
## Observations: 8328 | Variables: 11
```

```
str(wage[, c("wage", "experience", "age", "female", "univ")])
```

```
## 'data.frame':    8328 obs. of  5 variables:
## $ wage      : num  17.26 16.38 8.15 43.15 12.79 ...
## $ experience: num  18.12 20.88 0.75 12.88 4.79 ...
## $ age       : int   56 56 27 38 27 37 55 49 31 39 ...
## $ female    : int    1 1 0 1 0 0 1 1 0 1 ...
## $ univ      : int    1 0 1 1 0 1 0 1 1 1 ...
```

We regress log hourly wage on experience, age and an indicator for female, a simple Mincer-style specification.

```
wage$log_wage <- log(wage$wage)
fit_wage <- lm(log_wage ~ experience + age + female, data = wage)
diag_wage <- new_diag_lm(fit_wage, data_type = "cs")
kable(round(diag_wage$coefficients, 4), caption = "Wage regression: OLS coefficients")
```

Table 10: Wage regression: OLS coefficients

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	2.2656	0.0326	69.4123	0
experience	0.0094	0.0012	7.9945	0
age	0.0108	0.0011	9.6609	0
female	-0.1514	0.0149	-10.1303	0

Table 11: Wage regression: assumption diagnostics

Test	Value
Breusch-Pagan	stat = 62.837, p = 0.0000
Durbin-Watson	stat = 2.015, p = 0.7532
Shapiro-Wilk	p = NA
Max VIF	4.131

```
print(plot(diag_wage, type = "residuals"))
print(plot(diag_wage, type = "qq"))
```

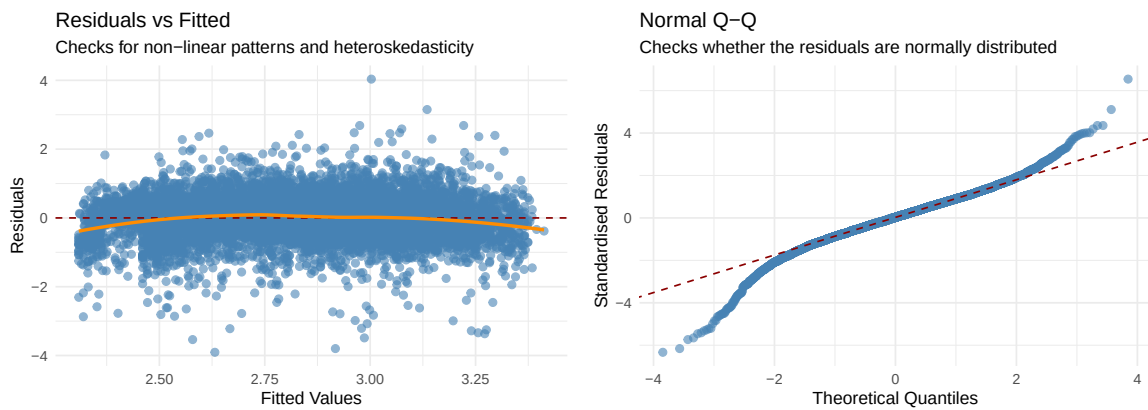


Figure 7: Wage regression diagnostics: Residuals vs Fitted (left) and Normal Q-Q (right).

Warning: Heteroskedasticity detected. Consider using robust standard errors.

Real survey data on wages routinely departs from the idealised simulation scenarios above (e.g. wages are typically right-skewed, which is why the model is fit on the log scale here), illustrating why an automated diagnostic layer is valuable in practice: a user can immediately see which assumptions still warrant attention even after a standard transformation.

5.2 Logistic Case Study: Predicting Sex from Labour-Market Characteristics

To exercise the logistic pathway on the same real dataset, we fit `female` (a 0/1 indicator already present in `data/wage.csv`) on age and experience, a purely illustrative specification chosen only to demonstrate the diagnostic workflow on non-simulated data, not a substantive labour- economics claim.

```
fit_wage_logit <- glm(female ~ age + experience, data = wage, family = binomial())
diag_wage_logit <- new_diag_lm(fit_wage_logit, data_type = "logistic")
kable(round(diag_wage_logit$coefficients, 4), caption = "Wage data logistic regression: c
```

Table 12: Wage data logistic regression: coefficient estimates

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-3.3858	0.1285	-26.3451	0
age	0.1425	0.0045	31.6029	0
experience	-0.1685	0.0048	-35.0929	0

Table 13: Wage data logistic regression: diagnostics

Test	Value
Hosmer-Lemeshow	p = 0.0000
McFadden pseudo-R2	0.1456
AUC	0.7457
Accuracy (0.5 cutoff)	0.6843
Max VIF	5.019

```
print(plot(diag_wage_logit, type = "roc"))
print(plot(diag_wage_logit, type = "binned_residuals"))
```

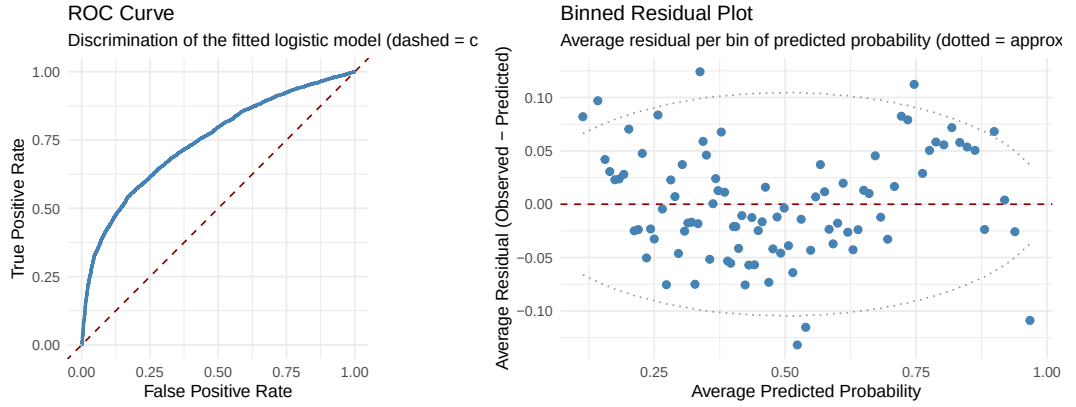


Figure 8: Wage data logistic regression: ROC curve (left) and Binned Residual plot (right).

Warning: Hosmer-Lemeshow test rejects adequate fit. Consider adding non-linear terms or interactions.

As expected for a weak, purely illustrative specification with only two predictors, discrimination is modest; the case study nonetheless confirms that the full logistic pipeline (fitting, all five diagnostics, three plot types and rule-based remediation) runs end-to-end on real, non-simulated survey data exactly as it does in the controlled simulations of Section 4.

6 The Shiny Application

`app.R` is the primary application (Section 2.4) and presents the workflow above through three tabs. **Setup** lets the user choose a **Model Family** (Linear/OLS or Logistic/Binomial), a data source (simulate, a bundled wage dataset or an uploaded CSV), and, for linear models, the econometric structure (cross-section, time series or panel); it also hosts an in-app variable-type editor. **Diagnostics** reports the fitted model, colour-coded PASS/CHECK/CAUTION status indicators and a set of diagnostic plots (both the value boxes and the plot grid are generated dynamically via `renderUI()`) so that only diagnostics relevant to the selected model family and data structure are shown (Section 2.4). **Transformation** supports interaction-term construction and log/exp/square/standardise/log-return column transforms. The screenshots below (captured live from a `shiny::runApp()` session during the preparation of this report) show the running application across the linear cross-sectional, logistic and time-series pathways.

Data Setup

Model Family:
☒ Linear (OLS) ☐ Logistic (Binomial)

Data Source:
☒ Simulate data
☐ Choose available data
☐ Upload a CSV

Select Econometric Data Structure:
 Cross-Sectional

Simulate a dataset to test OLS assumptions.

Sample Size (n):
 50 140 240 340 440 500 600 710 810 900 1,000

☐ Inject Heteroskedasticity
☐ Inject Multicollinearity

Random seed (leave blank for a new draw):

Simulate & Load Framework

Modify Variable Types
 Select variable to edit:
 Y

New data type:
 Factor

Convert Data Type

Data Preview & Statistical Properties

System Metadata & Structural Profile
 Data Structure Class: Cross-Sectional
 Observations Count (N): 500
 Total Variables Shown: 3

Raw Data Grid

	Y	X1	X2
13.94	6.16	8.00	
10.75	7.57	11.41	
15.80	7.02	8.09	
-4.62	0.92	10.93	
10.54	6.25	9.69	
-2.75	3.63	15.27	
-5.22	3.12	13.26	
3.20	4.97	14.23	
-0.75	2.29	10.12	
0.85	3.66	12.77	
5.25	5.87	14.87	
-11.98	1.12	14.93	

Data Column Profiles

No	Variable	Stats / Values	Freqs (% of Valid)	Missing
1	Y [numeric]	Mean (sd) : 7.9 (7.3) min <= med <= max: -12.0 <= 0.0 <= 29.9 IQR (CV) : 9.0 (0.9)	500 distinct values	0 (0.0%)
2	X1 [numeric]	Mean (sd) : 5.0 (2.8) min <= med <= max: -0.8 <= 5.0 <= 11.2 IQR (CV) : 2.6 (0.4)	500 distinct values	0 (0.0%)
3	X2 [numeric]	Mean (sd) : 9.8 (3.0) min <= med <= max: 1.5 <= 9.7 <= 17.0 IQR (CV) : 4.2 (0.3)	500 distinct values	0 (0.0%)

Figure 9: Setup tab (Linear model family): choosing a data source, econometric structure and sample size for the simulator.

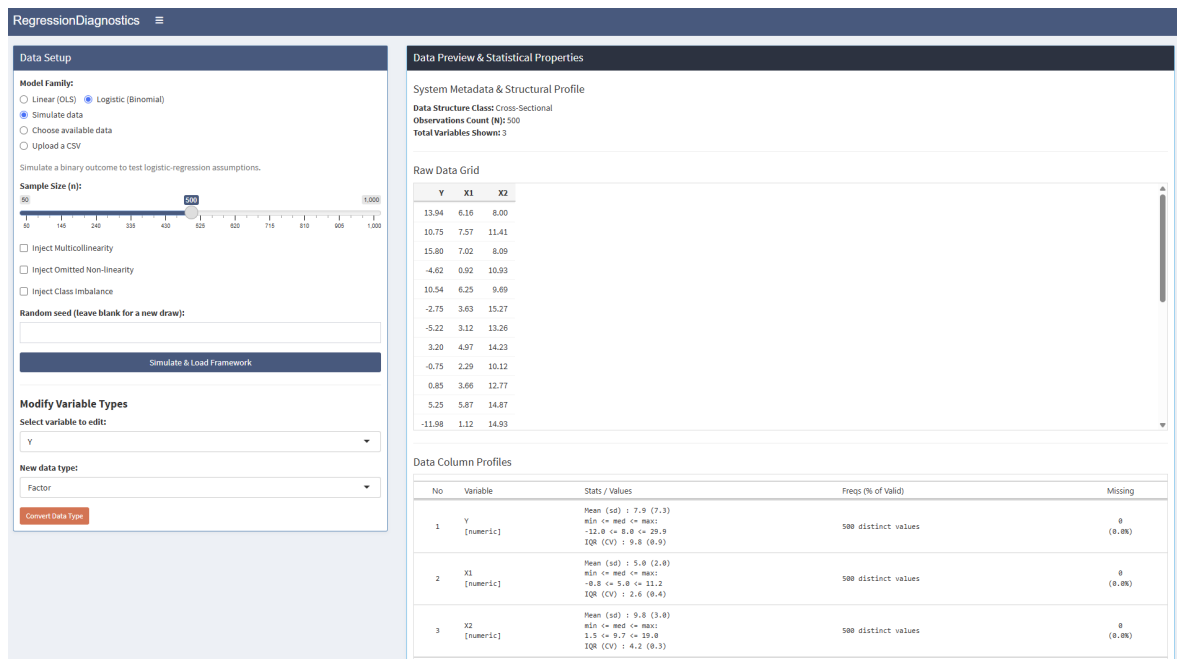


Figure 10: Setup tab switched to the Logistic model family: the violation controls change to Inject Multicollinearity / Omitted Non-linearity / Class Imbalance, matching `simulate_logit_data()`.

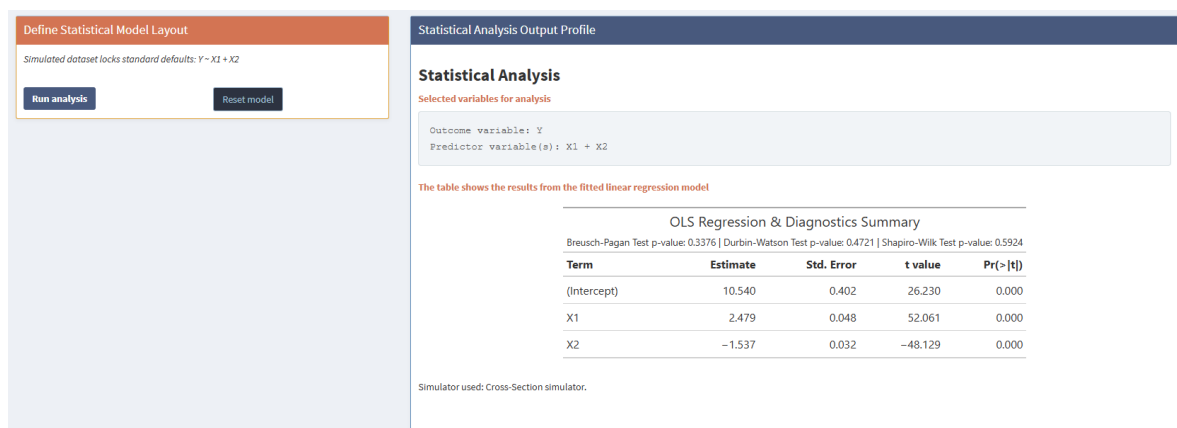


Figure 11: Diagnostics tab (Linear, cross-section): fitted coefficients and the Breusch-Pagan / Durbin-Watson / Shapiro-Wilk p-values embedded in the table subtitle.

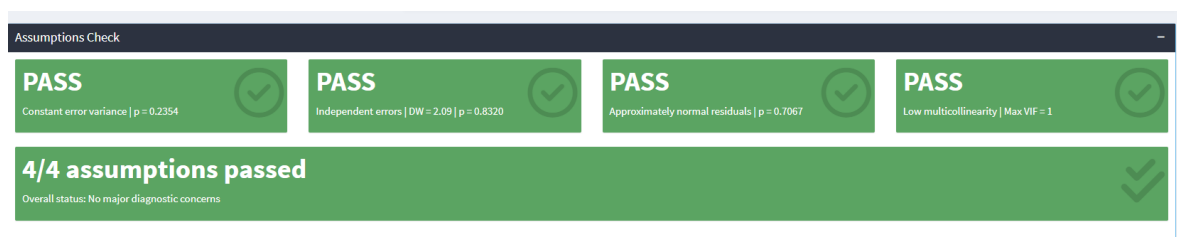


Figure 12: Diagnostics tab (Linear, cross-section): colour-coded PASS indicators for each of the four core assumption checks.



Figure 13: Diagnostics tab (Linear, cross-section): the four standard diagnostic plots (Residuals vs Fitted, Normal Q-Q, Scale-Location, Residual Histogram).

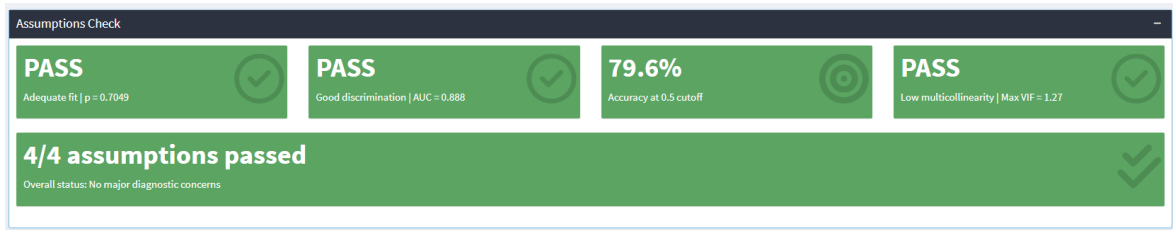


Figure 14: Diagnostics tab (Logistic): the Odds Ratio column and the Hosmer-Lemeshow / McFadden R² / AUC / Accuracy subtitle in the summary table.

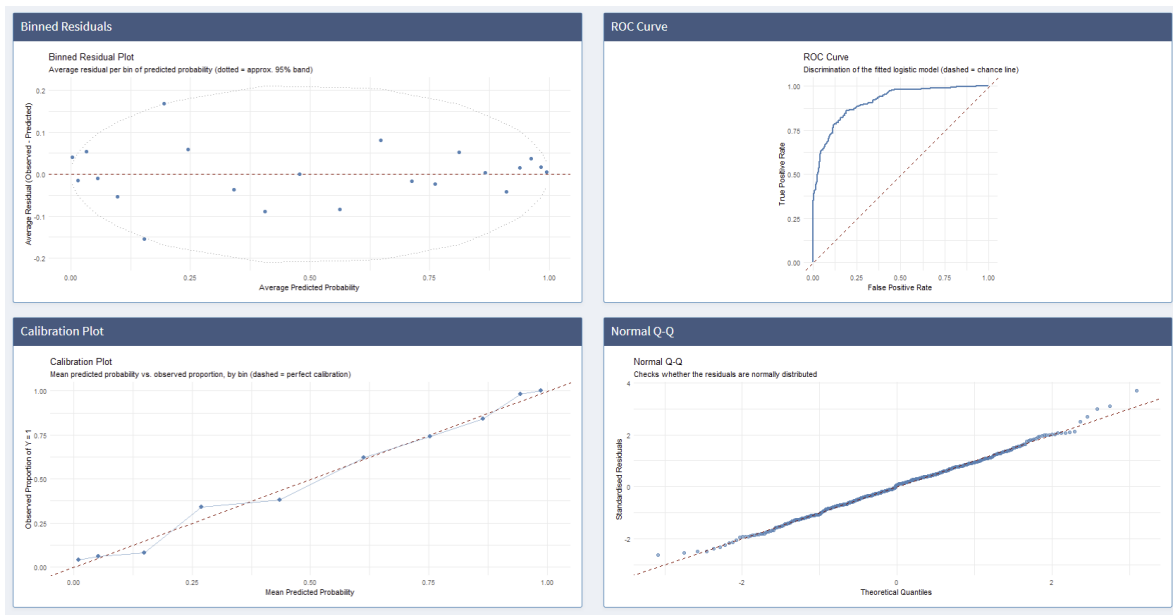


Figure 15: Diagnostics tab (Logistic): Binned Residuals, ROC Curve, Calibration Plot and Normal Q-Q, the plot menu curated for a binomial model.

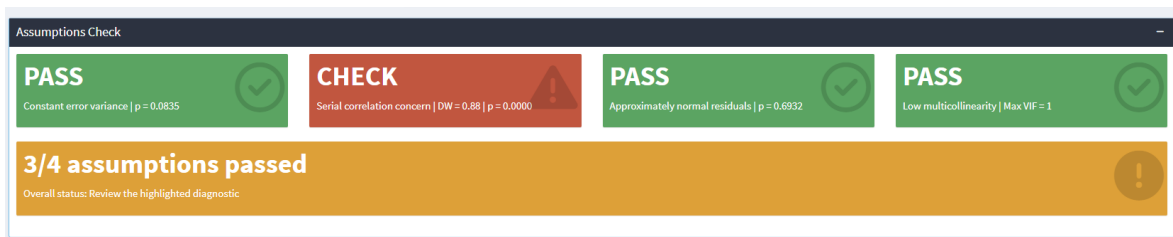


Figure 16: Diagnostics tab (Linear, time series): a deliberately “failing” run: the value boxes correctly flag serial correlation ($DW = 0.84$) and non-normality, while the curated Residuals-over-Time and ACF plots show the persistent pattern responsible.



Figure 17: Diagnostics tab (Linear, time series): Residuals-over-Time (left) and ACF (right), replacing the fitted-value-based plots that are not designed to reveal serial correlation.

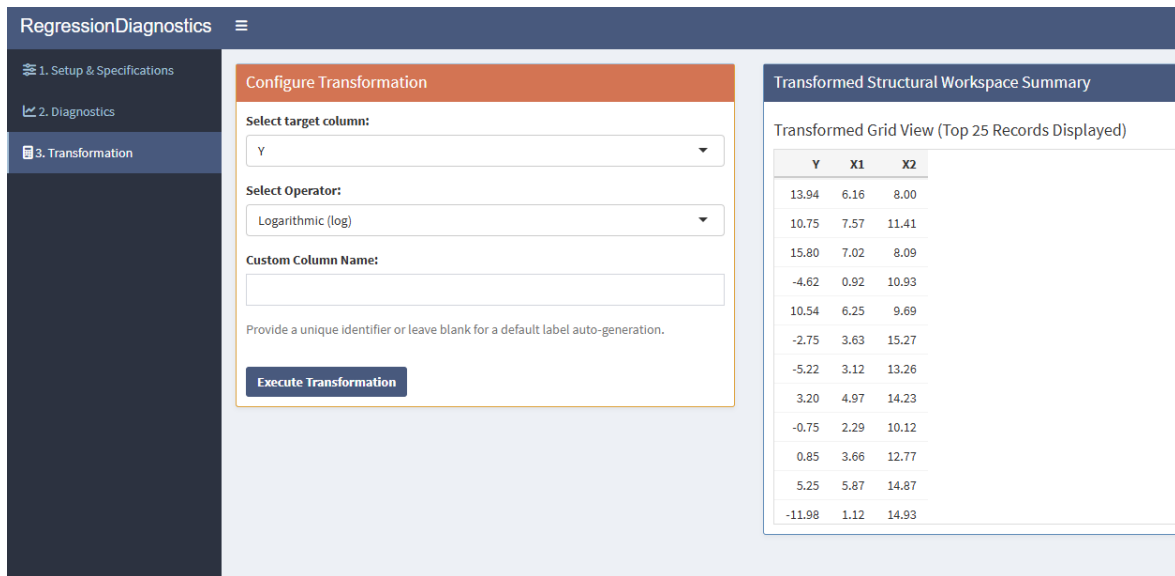


Figure 18: Transformation tab: applying a log/exp/square/standardise/log-return transform to a chosen column.

6.1 AI-Assisted Interpretation (Optional, Opt-In)

The Diagnostics tab also offers an optional “AI Interpretation” panel, implemented in `R/llm_interpret.R` and wired into `app.R` as a purely additive feature that leaves every existing output untouched. Clicking **Generate AI Interpretation** sends a compact, privacy-safe summary of the *already-computed* diagnostic statistics (model type, coefficients and the relevant test statistics/p-values, never raw data rows) to a third-party large language model (Groq’s `openai/gpt-oss-120b`, via its OpenAI-compatible chat-completions API)

and renders the model's plain-language narrative alongside the deterministic, rule-based `remediation_advice()` output (Section 2.2). The call is strictly opt-in: nothing is sent until the button is clicked, and if no API key is configured the panel falls back to an explanatory message instead of failing, so the deterministic parts of the app and its reproducibility are unaffected whether or not this feature is used. Because the response originates from an external service, it is escaped and parsed through a Markdown-to-HTML pipeline (`commonmark`) before display, so any stray HTML in the returned text is neutralised rather than executed. This feature is disclosed here for transparency, consistent with the use of an external AI service being permitted for this assignment.

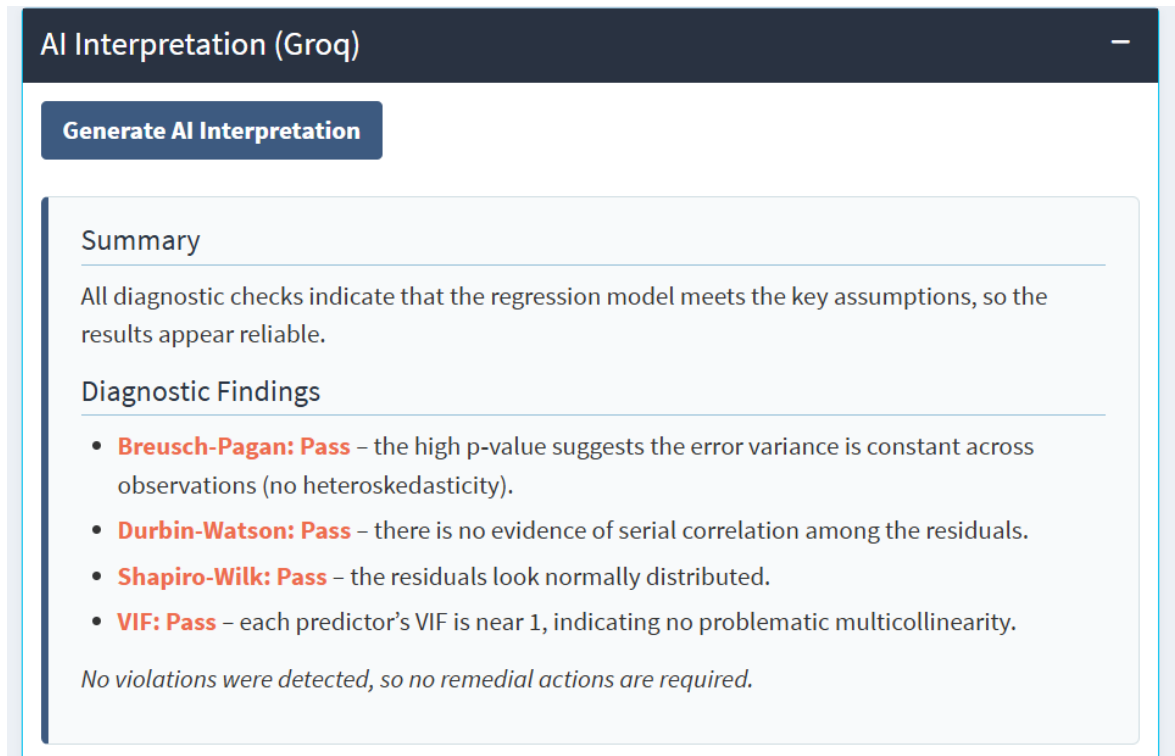


Figure 19: Diagnostics tab: the AI Interpretation panel after clicking Generate AI Interpretation, showing a structured Summary, per-test Diagnostic Findings and Recommended Actions generated from the already-computed diagnostics.

7 Testing and Validation

7.1 Unit-Level Tests of the `diag_lm` Class

`tests/test_diag_lm.R` is a lightweight, dependency-free smoke-test script (not a `testthat` suite, consistent with the project not being a formal package) that is executed below to demonstrate it passes as part of this report's own reproducible build.

```
old_wd <- setwd("../")
source("tests/test_diag_lm.R")
```

== Test 1: Multi-predictor model ==

Class check passed: diag_lm

Diagnostic Linear Model (diag_lm)

Formula: mpg ~ wt + hp + disp

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.1055	2.1108	17.5788	0.0000
wt	-3.8009	1.0662	-3.5649	0.0013
hp	-0.0312	0.0114	-2.7245	0.0110
disp	-0.0009	0.0103	-0.0905	0.9285

Diagnostics:

Breusch-Pagan statistic : 0.9459

Breusch-Pagan p-value : 0.8143 (no evidence of heteroskedasticity)

VIF scores :

	wt	hp	disp
	4.8446	2.7366	7.3245

== Test: summary() returns a gt table ==

summary() returned a gt_tbl object.

== Test: plot() returns a ggplot object ==

plot(type = residuals) returned a ggplot object.

plot(type = qq) returned a ggplot object.

plot(type = scale_location) returned a ggplot object.

plot(type = histogram) returned a ggplot object.

== Test: unknown plot type errors ==

Caught expected error: Error: Plot type 'qqplot' is not implemented yet.

== Test 2: Single-predictor model (VIF should be NA) ==

VIF correctly set to NA for single predictor.

== Test 3: Input validation (should error) ==

Caught expected error: Error: Input must be a fitted model of class 'lm' or 'plm'.

== Test 4: Logistic regression (glm binomial) ==

Class / data_type checks passed.

Logistic diagnostics (Hosmer-Lemeshow / AUC / McFadden R^2) computed.

`summary()` returned a `gt_tbl` object for the logistic model.

`plot(type = binned_residuals)` returned a `ggplot` object.

`plot(type = roc)` returned a `ggplot` object.

`plot(type = calibration)` returned a `ggplot` object.

== Test 5: Logistic plot types reject non-logistic models ==

Caught expected error: Error: The 'roc' plot is only available for logistic (binary) models

== Test 6: Time-series-only plot types (residuals_time / pacf) ==

`plot(type = residuals_time)` returned a `ggplot` object.

`plot(type = pacf)` returned a `ggplot` object.

All tests passed.

```
setwd(old_wd)
```

The suite checks: (i) that `new_diag_lm()` returns an object of class `diag_lm` for a valid multi-predictor model; (ii) that `summary()` returns a `gt_tbl` object; (iii) that `plot()` returns a `ggplot` object for each supported `type`, and raises an informative error for an unsupported type; (iv) that VIF is correctly set to `NA` for a single-predictor model (where it is mathematically undefined); (v) that the constructor rejects non-`lm`/`plm` input with an explicit error; (vi) for the logistic extension, that a binomial `glm` produces valid Hosmer-Lemeshow/AUC/McFadden R^2 diagnostics, a `gt_tbl` summary and working `binned_residuals`/`roc`/`calibration` plots; (vii) that those three logistic-only plot types raise an informative error when requested on a non-logistic model; and (viii) that the time-series-only `residuals_time`/`pacf` plot types render correctly.

7.2 Shiny-Level Integration Tests of `app.R`

Unit tests on `diag_lm` alone do not exercise the reactive graph that actually wires user input to model fitting and output rendering inside the `shiny` app. To close that gap, the application underwent a structured QA audit: the full server logic of `app.R` was mapped observer-by-observer and reactive-by-reactive, and every input path (model family switches, data upload vs. simulation, panel index selection, in-place type conversion) was checked against its downstream consumers. This audit surfaced five concrete defects, all of which were fixed in the current source (`app.R`, included in the project repository):

1. **Model-family race condition.** The logistic-only outputs (`hl_box`, `auc_box`, `accuracy_box`, `overall_box_logistic` and the `binned_residuals`/`roc`/`calibration` plots) read `diag_model()$diagnostics$logistic` without first confirming `diag_model()$data_type == "binary"`. Rapidly switching **Model Family** from

logistic to linear could leave one of these outputs bound to a non-logistic fit, producing an uncaught `Error in if: missing value where TRUE/FALSE needed`. Fixed by adding `req(identical(diag_model()$data_type, "binary"))` as the first line of each affected renderer.

2. **Formula-injection risk.** The model formula was built with `paste(y, "~", paste(x, collapse = " + "))` followed by `as.formula()`, which breaks on non-syntactic column names. Replaced with `reformulate(termlabels = ..., response = ...)`, which quotes names safely.
3. **Silent all-NA coercion.** Selecting a non-numeric column as the outcome or a predictor silently coerced it to all-NA via `suppressWarnings(as.numeric(...))`, yielding a meaningless fit with no warning to the user. Fixed with an explicit `validate(need(!all(is.na(...))))` check immediately after coercion, for the outcome and for each predictor.
4. **Blank panel index fields.** `req(input$p_idx_i, input$p_idx_t)` only guards against NULL, not against empty strings, letting blank index fields reach `plm` and fail with a cryptic internal error. Fixed with `validate(need(nzchar(...) && nzchar(...)))` plus an explicit column-existence check.
5. **Stale column reference in type conversion.** The in-place change-data-type feature did not check that the selected column still existed in the current data frame. Fixed with `validate(need(input$var_to_edit %in% names(df), ...))`.

To guard against regressions, these findings were codified as `tests/test_app_server.R`, a suite of nine `shiny::testServer()` tests that drive the real server function with simulated user input rather than testing `diag_lm` in isolation. It is executed below.

```
old_wd <- setwd("..")
source("tests/test_app_server.R")
```

```
== Test 1: Linear / Cross-Section default flow ==
```

```
PASS: linear/cs diag_model() is a valid cross-section diag_lm.
```

```
== Test 2: Logistic flow end-to-end ==
```

```
PASS: logistic diag_model() computed AUC = 0.908
```

```
== Test 3 (REGRESSION): model_family race condition (logistic -> linear) ==
```

```
PASS: logistic-only outputs no longer error when diag_model() is non-logistic.
```

```
== Test 4: Time-series flow with curated plots ==
```

PASS: time-series `diag_model()` + curated plots (`residuals_time`, `pacf`) work.

== Test 5: Panel data flow (uploaded CSV) ==

PASS: panel data flow computes Hausman $p = 0.8551$

== Test 6 (EDGE CASE): awkward original column names surviving `read.csv()` ==

PASS: sanitized column names (`Y..target.`, `X.1`, `X.2`) work end-to-end via `reformulate()`.

== Test 7 (EDGE CASE): non-numeric column selected as outcome ==

PASS: non-numeric outcome column is caught with a `validate()` message, not a silent bad fi

== Test 8 (EDGE CASE): blank panel index columns ==

PASS: blank panel index fields are rejected with a `validate()` message.

== Test 9 (EDGE CASE): type conversion on a non-existent column ==

PASS: converting a non-existent column does not crash the session.

ALL APP SERVER INTEGRATION TESTS PASSED

```
setwd(old_wd)
```

The nine tests cover, in order: (1) the default linear/cross-section flow; (2) an end-to-end logistic flow, including a working AUC computation; (3) a dedicated **regression test** reproducing defect 1 above by switching Model Family from logistic to linear and then force-evaluating the logistic-only outputs, confirming they now fail silently (`req()`'s intended behaviour) rather than erroring; (4) the time-series flow with its curated `residuals_time/pacf` plots; (5) an uploaded panel dataset, including a working Hausman test; (6) an uploaded CSV with awkward, non-syntactic original column names, confirming the `reformulate()`-based fix (defect 2) handles the names `read.csv()` sanitizes them into; (7) a non-numeric outcome column, confirming defect 3's `validate()` guard triggers; (8) blank panel index fields, confirming defect 4's guard triggers; and (9) an in-place type conversion targeting a column that no longer exists, confirming defect 5's guard prevents a session crash.

Together with the simulation study (Section 4) and the applied case studies (Section 5), this constitutes a four-tier validation strategy: unit-level (does the code do what it claims to do?), integration-level (does the actual app reactive graph behave correctly, including under adversarial and edge-case input?), statistical (does it recover known ground truth, for both model families?) and applied (does it behave sensibly on real, messy data?).

8 Discussion, Limitations and Future Work

Reliance on asymptotic tests. Breusch–Pagan, Durbin–Watson (as a p-value via `lmtest::dwtest`), the Breusch–Godfrey/Ljung–Box tests and the Hosmer–Lemeshow test all rely on large-sample approximations; in very small samples their nominal size may not hold exactly.

VIF and Shapiro–Wilk edge cases. VIF is undefined with a single predictor and Shapiro–Wilk becomes unreliable for very large ($n > 5000$) samples; both are guarded with `tryCatch()` in the current implementation, returning NA rather than an uninformative or misleading value.

Logistic scope is cross-sectional only. The current logistic extension supports `glm(family = binomial())` on cross-sectional data; panel logit (e.g. conditional or random-effects logit) and multinomial/ordinal outcomes are not yet supported, and would require separate estimation routines (e.g. via `pglm` or `nnet::multinom`) rather than a direct extension of the current `glm`-based pathway.

Separation heuristic is deliberately simple. The current (quasi-)complete-separation flag checks only for implausibly large standard errors; a more rigorous implementation would test the Albert–Anderson (Albert & Anderson, 1984) conditions directly, and automatically refit with Firth’s correction (Firth, 1993) rather than only recommending it.

No formal package structure. As noted throughout, the project is distributed as plain scripts. Converting R/ into an installable package (with a DESCRIPTION, NAMESPACE and roxygen2-generated documentation) would enable R CMD check, a real vignette() and easier reuse outside this specific shiny app; this is the most actionable piece of future work.

Remediation is advisory only. `remediation_advice()` currently only prints suggestions (e.g. “use robust standard errors”, “consider Firth’s correction”); a natural extension is to let the app automatically refit with sandwich-based heteroskedasticity- and autocorrelation-consistent (HAC) standard errors, or a penalised-likelihood logistic fit, when a violation is detected.

9 Conclusion

This report has documented a complete, reproducible pipeline for diagnosing violations of both the classical linear regression assumptions and the analogous fit/discrimination properties of a binomial logistic regression: a well-tested S3 class encapsulating diagnostic tests for four data structures (cross-section, time-series, panel and logistic), an interactive shiny application (`app.R`) that makes those diagnostics accessible without writing R code and that curates its value boxes and plots to the selected model family and data structure and a four-tier validation strategy spanning unit tests, Shiny-level integration tests, a controlled simulation study and real-world case studies. The simulation study confirmed that each implemented test responds correctly and selectively to the assumption violation it targets, for

both model families, and the wage-data case studies showed the framework behaving sensibly on real, non-simulated data. Because the project is not packaged formally, Appendix B supplies a vignette-style guide to the `diag_lm` class so that the documentation requirement is satisfied in full within this report.

Appendix A

A.1 Description of Relevant Variables

Table 14: Description of relevant variables

Variable	Dataset	Description
wage	wage.csv	Gross hourly wage in EUR, the outcome variable of the linear case study
log_wage	wage.csv (derived)	Natural logarithm of wage, the outcome actually modelled in Section 5.1
age	wage.csv	Respondent's age in years
experience	wage.csv	Years of labour-market experience
female	wage.csv	Indicator (1/0) for female respondents; the outcome of the logistic case study in Section 5.2
univ	wage.csv	Indicator (1/0) for holding a university degree
east	wage.csv	Indicator (1/0) for residing in East Germany
minijob	wage.csv	Indicator (1/0) for marginal ('mini-job') employment
apprent	wage.csv	Indicator (1/0) for holding a vocational apprenticeship qualification
novoc	wage.csv	Indicator (1/0) for holding no vocational qualification
limited	wage.csv	Indicator (1/0) for a fixed-term ('limited') employment contract
persnr	wage.csv	Anonymised respondent identifier, not used as a model predictor
Y	simulated (cross-section/time-series/logistic)	Simulated outcome variable generated by <code>simulate_ols_data()/simulate_ts_data()/simulate_logit_data()</code> (Section 4)
X1	simulated (cross-section/time-series/logistic)	First simulated predictor, correlated with the outcome by construction
X2	simulated (cross-section/time-series/logistic)	Second simulated predictor, optionally made collinear with X1 to inject multicollinearity
t	simulated (time-series)	Time index (1..n) attached to the simulated time-series observations
id	simulated (panel, vignette)	Cross-sectional (individual) identifier for the synthetic panel example in Appendix B
time	simulated (panel, vignette)	Time identifier for the synthetic panel example in Appendix B

Appendix B: Vignette-Style Usage Guide for `diag_lm`

This project is not distributed as a formal, installable R package (there is no `DESCRIPTION` or `NAMESPACE` file), so a package vignette in the strict sense does not apply. This appendix nonetheless provides the equivalent narrative, end-to-end documentation — in the same spirit and depth as an R package vignette built with `knitr/rmarkdown` — so that the documentation requirement is fully satisfied within this report.

9.1 Installation and Setup

The `diag_lm` class has no installation step of its own; it is used by sourcing the script directly, together with its package dependencies:

```
install.packages(c("shiny", "shinydashboard", "lmtest", "car", "plm",
                  "gt", "ggplot2", "dplyr", "tibble"))
source("R/diag_lm_class.R")
source("R/simulate_data.R")
```

9.2 Minimal Worked Example

```
set.seed(1)
demo_data <- simulate_ols_data(n = 150, heteroskedastic = FALSE, multicollinear = FALSE)
demo_fit <- lm(Y ~ X1 + X2, data = demo_data)
demo_diag <- new_diag_lm(demo_fit, data_type = "cross-section")

class(demo_diag)
```

```
## [1] "diag_lm"
```

9.2.1 `print()`: Console Overview

```
print(demo_diag)
```

Diagnostic Linear Model (diag_lm)

Formula: Y ~ X1 + X2

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	9.9448	0.7637	13.0219	0

X1	2.5036	0.0938	26.7007	0
X2	-1.4975	0.0553	-27.0959	0

Diagnostics:

Breusch-Pagan statistic : 0.7476

Breusch-Pagan p-value : 0.6881 (no evidence of heteroskedasticity)

VIF scores :

X1	X2
----	----

1.0011	1.0011
--------	--------

9.2.2 summary(): Formatted Diagnostic Table

`summary.diag_lm()` returns a `gt` table (rendered as a plain data frame here for print purposes) whose header reports the Breusch–Pagan, Durbin–Watson, and Shapiro–Wilk p-values alongside the coefficient table:

```
kable(round(demo_diag$coefficients, 4), caption = "Demo model: OLS coefficient table (vignette example)")
```

Table 15: Demo model: OLS coefficient table (vignette example)

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	9.9448	0.7637	13.0219	0
X1	2.5036	0.0938	26.7007	0
X2	-1.4975	0.0553	-27.0959	0

```
cat(sprintf(
  "Breusch-Pagan p = %.4f | Durbin-Watson p = %.4f | Shapiro-Wilk p = %.4f\n",
  demo_diag$diagnostics$bp_pvalue,
  demo_diag$diagnostics$dw_pvalue,
  demo_diag$diagnostics$shapiro_pvalue
))
```

```
## Breusch-Pagan p = 0.6881 | Durbin-Watson p = 0.2606 | Shapiro-Wilk p = 0.1048
```

9.2.3 plot(): The Five Supported Diagnostic Views

```
print(plot(demo_diag, type = "residuals"))
print(plot(demo_diag, type = "qq"))
print(plot(demo_diag, type = "scale_location"))
```

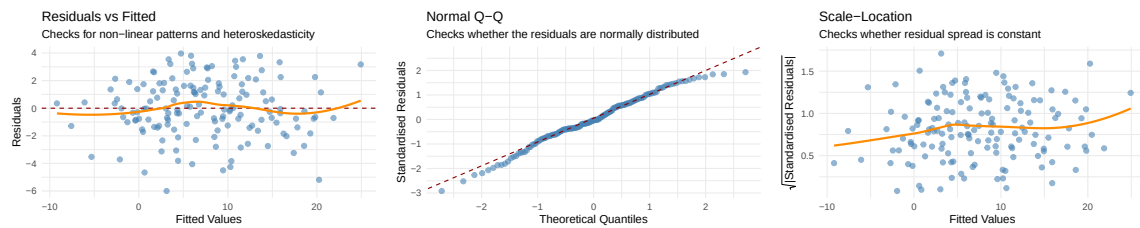


Figure 20: All four cross-sectional plot types supported by `plot.diag-lm()`, left to right: residuals, qq, scale-location, histogram.

```
print(plot(demo_diag, type = "histogram"))
```

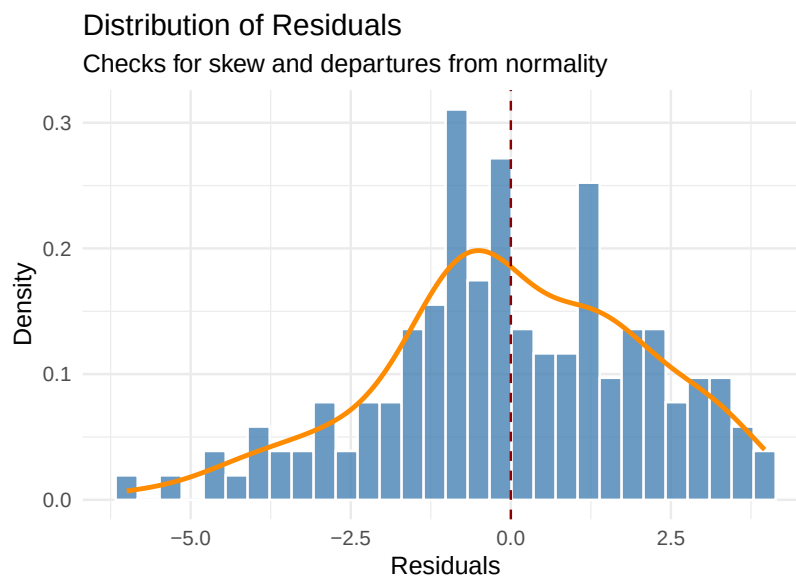


Figure 21: Residual histogram (fourth cross-sectional plot type).

Requesting an unsupported `type` raises an informative error rather than failing silently:

```
plot(demo_diag, type = "not_a_real_type")
```

```
## Error:
```

```
## ! Error: Plot type 'not_a_real_type' is not implemented yet.
```

9.2.4 `remediation_advice()`: Rule-Based Guidance

```
set.seed(2)
bad_data <- simulate_ols_data(n = 150, heteroskedastic = TRUE)
bad_fit <- lm(Y ~ X1 + X2, data = bad_data)
bad_diag <- new_diag_lm(bad_fit, data_type = "cross-section")
cat(gsub("<[>]+>", "", remediation_advice(bad_diag)))
```

```
## Warning: Heteroskedasticity detected. Consider using robust standard errors. Warning:
```

9.2.5 Working with Panel Data

`diag_lm` also supports pooled panel models fitted with `plm::plm()`. The example below constructs a small synthetic panel with individual fixed effects and lets `new_diag_lm()` run the panel Breusch–Pagan and Hausman tests automatically:

```
set.seed(3)
n_id <- 30; n_time <- 5
panel_df <- expand.grid(id = 1:n_id, time = 1:n_time)
alpha_i <- rep(rnorm(n_id, 0, 2), each = n_time)
panel_df$X <- rnorm(nrow(panel_df), 5, 2)
panel_df$Y <- 3 + 1.5 * panel_df$X + alpha_i + rnorm(nrow(panel_df), 0, 1)

pooled_fit <- plm::plm(Y ~ X, data = panel_df, model = "pooling", index = c("id", "time"))
panel_diag <- new_diag_lm(pooled_fit, data_type = "panel")

cat(sprintf("Panel BP p-value:      %.5f\n", panel_diag$diagnostics$panel$bp_pvalue))
```

```
## Panel BP p-value:      0.84882
```

```
cat(sprintf("Hausman test p-value:  %.5f\n", panel_diag$diagnostics$panel$hausman_pvalue))
```

```
## Hausman test p-value:  0.86084
```

Because the synthetic panel contains genuine, persistent individual effects (α_i), the panel Breusch–Pagan test correctly detects their presence, and the significant Hausman test correctly favours a fixed-effects specification over the naive pooled model.

9.2.6 Working with Logistic Regression

`diag_lm` also supports binomial `glm` models via `data_type = "logistic"` (alias `"binary"`). The example below simulates a binary outcome, fits a logistic regression and inspects the logistic-specific diagnostics and plot types:

```
set.seed(4)
logit_demo_data <- simulate_logit_data(n = 300, multicollinear = FALSE,
                                       nonlinear = FALSE, imbalance = FALSE)
logit_demo_fit <- glm(Y ~ X1 + X2, data = logit_demo_data, family = binomial())
logit_demo_diag <- new_diag_lm(logit_demo_fit, data_type = "logistic")
```

```
cat(sprintf(
  "Hosmer-Lemeshow p = %.4f | McFadden R2 = %.4f | AUC = %.4f | Accuracy = %.4f\n",
  logit_demo_diag$diagnostics$logistic$hl_pvalue,
  logit_demo_diag$diagnostics$logistic$mcfadden_r2,
  logit_demo_diag$diagnostics$logistic$auc,
  logit_demo_diag$diagnostics$logistic$accuracy
))
```

```
## Hosmer-Lemeshow p = 0.5774 | McFadden R2 = 0.3873 | AUC = 0.8858 | Accuracy = 0.8067
```

```
print(plot(logit_demo_diag, type = "binned_residuals"))
print(plot(logit_demo_diag, type = "roc"))
print(plot(logit_demo_diag, type = "calibration"))
```

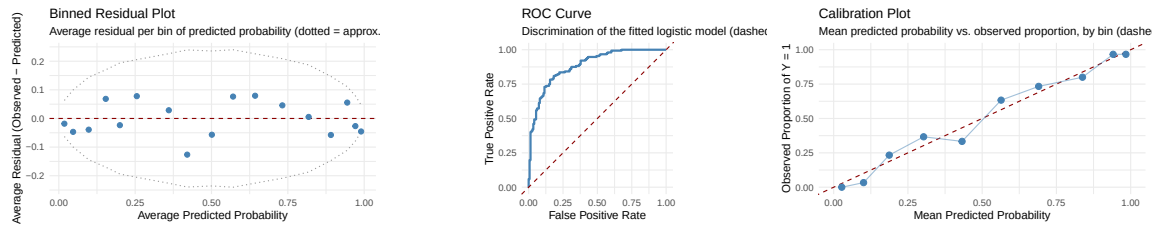


Figure 22: The three logistic-only plot types supported by `plot.diag-lm()`, left to right: binned-residuals, roc, calibration.

Requesting a logistic-only plot type on a linear model (or vice versa) raises an informative error rather than producing a nonsensical plot:

```
plot(demo_diag, type = "roc")
```

```
## Error:
```

```
## ! Error: The 'roc' plot is only available for logistic (binary) models.
```

9.3 Summary of Exported Functionality

Table 16: Summary of the `diag_lm` public API (vignette-style reference table)

Object	Description
<code>new_diag_lm()</code>	Constructor: fit diagnostics on an <code>lm</code> / <code>plm</code> / <code>binomial-glm</code> object
<code>print()</code>	S3 method: console summary (linear- or logistic-specific)
<code>summary()</code>	S3 method: gt-formatted diagnostic table (adapts to <code>data_type</code>)

Object	Description
plot()	S3 method: one of 10 diagnostic ggplot views (linear + logistic + time-series)
remediation_advice()	\$3 generic: rule-based HTML remediation messages
simulate_ols_data()	Cross-sectional data generator with optional violations
simulate_ts_data()	Time-series data generator with AR(1) errors and optional violations
simulate_logit_data()	Binary-outcome data generator with optional multicollinearity/non-linearity/imbalance

References

- Albert, A., & Anderson, J. A.** (1984). On the existence of maximum likelihood estimates in logistic regression models. *Biometrika*, *71*(1), 1–10.
- Breusch, T. S., & Pagan, A. R.** (1979). A simple test for heteroscedasticity and random coefficient variation. *Econometrica*, *47*(5), 1287–1294.
- Cox, D. R., & Snell, E. J.** (1989). *Analysis of binary data* (2nd ed.). Chapman; Hall.
- Durbin, J., & Watson, G. S.** (1950). Testing for serial correlation in least squares regression: I. *Biometrika*, *37*(3/4), 409–428.
- Durbin, J., & Watson, G. S.** (1951). Testing for serial correlation in least squares regression. ii. *Biometrika*, *38*(1/2), 159–177.
- Firth, D.** (1993). Bias reduction of maximum likelihood estimates. *Biometrika*, *80*(1), 27–38.
- Fox, J., & Weisberg, S.** (2019). *An r companion to applied regression* (3rd ed.). Sage Publications.
- Godfrey, L. G.** (1978). Testing against general autoregressive and moving average error models when the regressors include lagged dependent variables. *Econometrica*, *46*(6), 1293–1301.
- Gujarati, D. N., & Porter, D. C.** (2009). *Basic econometrics* (5th ed.). McGraw-Hill.
- Hanley, J. A., & McNeil, B. J.** (1982). The meaning and use of the area under a receiver operating characteristic (roc) curve. *Radiology*, *143*(1), 29–36.
- Hausman, J. A.** (1978). Specification tests in econometrics. *Econometrica*, *46*(6), 1251–1271.
- Hosmer, D. W., & Lemeshow, S.** (1980). A goodness-of-fit test for the multiple logistic regression model. *Communications in Statistics – Theory and Methods*, *9*(10), 1043–1069.
- Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X.** (2013). *Applied logistic regression* (3rd ed.). Wiley.

- Ljung, G. M., & Box, G. E. P.** (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303.
- McFadden, D.** (1974). Conditional logit analysis of qualitative choice behavior (**P. Zarembka**, Ed.), 105–142.
- Nagelkerke, N. J. D.** (1991). A note on a general definition of the coefficient of determination. *Biometrika*, 78(3), 691–692.
- Shapiro, S. S., & Wilk, M. B.** (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3/4), 591–611.
- Wooldridge, J. M.** (2020). *Introductory econometrics: A modern approach* (7th ed.). Cengage Learning.

Eidesstattliche Versicherung

Ich versichere an Eides statt durch meine Unterschrift, dass ich die vorstehende Arbeit selbstständig und ohne fremde Hilfe angefertigt und alle Stellen, die ich wörtlich oder annähernd wörtlich aus Veröffentlichungen entnommen habe, als solche kenntlich gemacht habe, mich auch keiner anderen als der angegebenen Literatur oder sonstiger Hilfsmittel bedient habe. Die Arbeit hat in dieser oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen.

Essen, den _____

Rezaul Hoque, Sarah Al-Talal, Thi Hong Cam Nguyen